Reflection and Traceability Report on Crazy 1-0s Game

Team #25, The Crazy Four
Ruida Chen
Ammar Sharbat
Alvin Qian
Jiaming Li

[Reflection is an important component of getting the full benefits from a learning experience. Besides the intrinsic benefits of reflection, this document will be used to help the TAs grade how well your team responded to feedback. Therefore, traceability between Revision 0 and Revision 1 is and important part of the reflection exercise. In addition, several CEAB (Canadian Engineering Accreditation Board) Learning Outcomes (LOs) will be assessed based on your reflections. —TPLT]

1 Changes in Response to Feedback

[Summarize the changes made over the course of the project in response to feedback from TAs, the instructor, teammates, other teams, the project supervisor (if present), and from user testers. —TPLT]

Our team made changes regularly over the course of the project in response to feedback from our instructors, supervisor and other teams (peer reviewers). However, these changes were generally reflected in our decision making as the project progressed. The changes were reflected in our docs and code only when the deadlines for these things came to fruition ...

[For those teams with an external supervisor, please highlight how the feedback from the supervisor shaped your project. In particular, you should highlight the supervisor's response to your Rev 0 demonstration to them. —TPLT]

We had 2 follow up meetings with Supervisor Rapoport after our Rev0 to further demonstrate our progress with the game and receive feedback.
This is the first meeting with supervisor Rapoport , and this is the second meeting with supervisor Rapoport.

...

[Version control can make the summary relatively easy, if you used issues and meaningful commits. If you feedback is in an issue, and you responded in the issue tracker, you can point to the issue as part of explaining your changes. If addressing the issue required changes to code or documentation, you can point to the specific commit that made the changes. Although the links are helpful for the details, you should include a label for each item of feedback so that the reader has an idea of what each item is about without the need to click on everything to find out. —TPLT]

[If you were not organized with your commits, traceability between feedback and commits will not be feasible to capture after the fact. You will instead need to spend time writing down a summary of the changes made in response to each item of feedback. —TPLT]

[You should address EVERY item of feedback. A table or itemized list is recommended. You should record every item of feedback, along with the source of that feedback and the change you made in response to that feedback. The response can be a change to your documentation, code, or development process. The response can also be the reason why no changes were made in response to the feedback. To make this information manageable, you will record the feedback and response separately for each deliverable in the sections that follow. —TPLT]

We did not save all of the Rev0 feedback in issues (did we?), but we did save the Rev0 feedback in a folder in our repo, viewable in the /docs/projMngmnt/Rev0$_Feedbackfolder$. From this feedback, along with Supervisor Rapoport's feedback, we made several changes to our code or development process. We have listed these changes below, along with the issue and/or commit the change was added in:

Feedback Item	**Response**	**Response Issue and/or Commit**

[If the feedback is general or incomplete, the TA (or instructor) will not be able to grade your response to feedback. In that case your grade on this document, and likely the Revision 1 versions of the other documents will be low. —TPLT]

1.1 SRS and Hazard Analysis

For SRS, there was not any instructor feedback on Avenue. The feedback we received was mostly from our peer review team. The issues which were closed for SRS were the following, and have been mapped to their resolving commits:

ID	Source	Feedback	Response / Change
S1	Peer Review	Stakeholders Issue #51	Updated stakeholders section. Commit 7b10ce0, closed in 60a49ab.
S2	Peer Review	Stakeholders Issue #52	Updated stakeholders section. Commit 7b10ce0, closed in 60a49ab.
S3	Peer Review	Mandated constraints (Implementation details) Issue #53	Updated mandated constraints for less implementation details. Commit 60a49ab.
S4	Peer Review	Mandated constraints Issue #54	Updated mandated constraints for less implementation details. Commit 60a49ab.
S5	Peer Review	Mandated constraints Issue #55	Updated mandated constraints for less implementation details. Commit 60a49ab.
S6	Peer Review	Relevant facts and assumptions Issue #56	Updated relevant fact and assumptions. Commit 4122fa0.
S7	Peer Review	Relevant facts and assumptions Issue #57	Updated relevant fact and assumptions. Commit 4122fa0.
S8	Peer Review	Scope of the work Issue #58	Updated scope of the work. Commit e46ef63.
S9	Peer Review	Scope of the work Issue #59	Updated scope of the work. Commit e46ef63.
S10	Peer Review	Goals and waiting room Issue #63	Updated goals and waiting room. Commit d88d6e4.
S11	Peer Review	Purpose of project Issue #65	Updated purpose of project and usability and humanity requirements. Commit 48dde45.

ID	Source	Feedback	Response / Change
S12	Peer Review	Usability and humanity requirements Issue #66	Updated usability and humanity requirements. Commit 48dde45.
S13	Peer Review	General SRS updates Issue #73	Addressed alongside the overarching structural changes to SRS sections. Commits d88d6e4 & 48dde45.

For the Hazard Analysis, feedback was provided during peer reviews and by TA Chris. The main concerns involved the completeness of the Failure Mode and Effects Analysis (FMEA) and the handling of data security, given the system's shift to a full web deployment with a database.

ID	Source	Feedback	Response / Change
H1	TA Chris (Rev 0)	"FMEA table lacks severity and occurrence ratings for several software hazards. Need to quantify the risks."	Updated the FMEA table to include standardized severity (1-10) and occurrence (1-10) ratings for all identified hazards, allowing for proper RPN (Risk Priority Number) calculation.
H2	Peer Review	Security hazards related to user authentication and database injection are missing from the analysis.	Added new hazard entries specifically for SQL injection and session hijacking. Added corresponding safety requirements to enforce parameterized queries and secure token storage.
H3	Instructor (Rev 0)	The hazard analysis does not adequately address the risk of incorrect educational content (e.g., faulty dozenal math logic).	Created a new hazard category for "Pedagogical Failure." Implemented strict unit testing for the base conversion module to mitigate the risk of teaching incorrect arithmetic.

1.2 Design and Design Documentation

The Design Document received two rounds of feedback: Rev −1 (Dec 8, 2025, 77.27%) and Rev 0 (Mar 5, 2026, 67.86%). The Rev 0 Demo (Feb 13, 2026)

also produced significant design feedback from the instructor.

ID	Source	Feedback	Response / Change
D1	TA Chris (Rev −1, Dec 8)	"No anticipated or likely changes listed. No uses hierarchy." MG quality scored 2/3.	Added anticipated and likely changes sections. Added uses hierarchy DAG diagram. Ruida expanded the timeline section .
D2	TA Chris (Rev −1)	"Should not mention things like 'React' in the design — that is an implementation detail."	Revised MG and MIS to use implementation-independent language. References to React, Express, PostgreSQL were abstracted to generic module descriptions.
D3	TA Chris (Rev −1)	M20, M21, M22 feel too low-level (process scheduling, DOM rendering, physical storage). M21/M22 are software hiding, not hardware hiding.	Those modules were removed, the whole MIS were re-designed.
D4	TA Chris (DesDoc, Mar 5)	"Timeline is not detailed. Should have information about when each module will be implemented by." Timeline scored 0.5/3.	Ruida expanded the timeline to per-module granularity with owner assignments and target dates (commit `aa3a601`).
D5	TA Chris (DesDoc, Mar 5)	"UCs are not labeled." Several SW decision modules misclassified (matchmaking, lobby view, game board view).	Labeled all Unlikely Changes with unique identifiers (UC1–UCn). Reclassified application-specific modules out of the generic software decision category.
D6	TA Chris (MIS)	MIS too low-level in places: "socket" in 12.4.1, "JSX" in 13.4.4, use `out :=` and `exc :=` notation, `void` is not mathematical.	Revised MIS semantics to use descriptive specification with `out :=` / `exc :=` notation. Removed JSX, socket, and void references. Fixed notation table (commit `64538fe`).

1.3 VnV Plan and Report

The Verification and Validation (V&V) Plan and Report received feedback during the Rev 0 and Rev 1 evaluation phases, primarily focusing on test coverage,

automated testing integration, and traceability.

ID	Source	Feedback	Response / Change
V1	Peer Review (Issues #91–#99)	Inconsistencies between SRS requirements and VnV Plan test cases. Some functional requirements lacked corresponding tests.	Added missing test cases to the VnV Plan. Ensured 1-to-1 traceability between SRS requirements and test cases in the Verification Plan.
V2	TA Chris (Rev 0)	"Unit testing strategy is not clearly defined. Need to specify what framework will be used and how coverage will be measured."	Updated the VnV Plan to explicitly specify Vitest for frontend/backend unit testing. Added coverage goals (e.g., 80% statement coverage).
V3	Instructor (POC / Rev 0)	"VnV Report lacks actual test execution results. The report should document the outcome of the planned tests, not just the plan."	Populated the VnV Report with execution results for Rev 0 and Rev 1. Added a section detailing automated test runs and manual usability findings.
V4	TA Chris (Rev 1)	"Usability testing methodology in VnV is vague. Need clearer metrics for non-functional requirements (NFRs)."	Expanded the NFR testing section. Integrated the formal Usability Testing extra (SUS scores, task completion rates) directly into the VnV validation results.
V5	Instructor (Final)	Ensure CI/CD pipeline correctly runs the automated tests described in the VnV Plan.	Configured GitHub Actions to automatically execute the Vitest suite on every PR, ensuring continuous verification of the codebase.

1.4 POC Demo, Rev 0 Demo and Final Demo

The POC Demo scored 73.33% (Nov 28, 2025). The Rev 0 Demo scored 14/21 (Feb 13, 2026). The Final Demo scored 18.5/27 (Apr 1, 2026).

ID	Source	Feedback	Response / Change
P1	Instructor (POC, Nov 28)	"I have concerns that the team is divided in two sets {Ammar, Dr. Rapoport} and {Ammar, Ruida, Jiaming, Alvin}, with Ammar as the only intersection. Going forward, everyone on the team should engage with Dr. Rapoport."	All team members participated in subsequent supervisor meetings. The Jan 22 in-person session and Feb 12 demo both had full attendance.
P2	Instructor (POC)	"The report makes it sound like CI/CD is implemented, but it is not. The CI/CD infrastructure should be built soon."	GitHub Actions CI/CD pipeline was implemented: automated builds, Vitest test execution, and LaTeX compilation on every PR.
P3	Instructor (POC)	"The team should have more team meetings to make sure you are on the same page." Team Communication score was mediocre.	Increased meeting frequency between POC and Rev0. Created GitHub Issues for future team meetings as suggested (Issue #262).
P4	Instructor (POC)	"A usability report should ask about more than whether the game is fun. The usability report should measure whether the game is effective at teaching different numerical bases."	Our usability study included tasks specifically measuring dozenal learning effectiveness (T14, T15) and custom Likert questions about educational value (Q1–Q5).
P5	Instructor (Rev0, Feb 13)	Rev0 difficulty scored 1.5/3. Team Communication scored 1/3. "The implementation should be further for Rev0 — authentication not set up, remove web socket connection, use actual cards, some unicode not displaying correctly."	Implemented authentication, concurrent games, actual card art, and full deployment for Rev1. Improved team coordination for Final Demo.

ID	Source	Feedback	Response / Change
P6	Prof. Rapoport (post-POC, Nov 26, Issue #133)	Features to implement: arithmetic problem on wildcard, bonus points for correct answers, timer on turns (possibly using the active number base), and multiplication operations.	Wildcard arithmetic system implemented in Rev1 per these suggestions. Suit-based operation triggers finalized in the Feb 12 meeting.
P7	Prof. Rapoport (Feb 12 meeting)	Detailed list of changes: starting deck 7 cards not 5, 10 card is wild (not Dek), arithmetic answers typed not selected, both players compete on challenges, goal 50 → 100, remove manual pass button, fix score displayed in decimal instead of dozenal, reshuffle discard pile when draw pile is depleted.	All items were implemented in Rev1.
P8	Prof. Rapoport (Apr 3 email)	Post-final feedback: only 7 cards dealt instead of 8, auxiliary tables use X/E instead of rotated 2/3, hand count in decimal not dozenal, missing round prompt after round 2, suit icons nearly invisible after wildcard, game didn't end at 100 dozenal points.	Hand size updated to 8. Hint tables rewritten with proper dozenal symbols. Hand count now in dozenal. Round prompt bug fixed. Suit visibility improved. Win condition fixed.
P9	Instructor (Final, Apr 1)	"Show a demo at the start. Rather than present MVP and stretch goals, show what you built. Too much time on project roadmap. If you are excited, the audience will be excited — convey more energy." Presentation scored 1.5/3.	Noted for future presentations. The team acknowledged insufficient rehearsal and energy.

ID	Source	Feedback	Response / Change
P10	Users from Usability Testing Report (Final)	"The illegal-move doesn't have a clear feedback such as a pop-up window', the error messages only appear in the collapsible activity logs, normal users wouldn't mention it.	Implemented error message pop-up windows, enhance user feedbacks.
P11	Instructor (Final)	"I like the instruction manual and the addition table. Deployment and devops are good." Final Demo scored 18.5/27.	Positive feedback confirming the rulebook and deployment infrastructure met expectations. Dr. Rapoport separately noted: "The game has made good progress, especially recently!"

2 Extras

[Summarize the extras that were tackled by this project. Extras can include usability testing, code walkthroughs, user documentation, formal proof, GenderMag personas, Design Thinking, etc. Extras should have already been approved by the course instructor as included in your problem statement. —TPLT]

Our team completed two extras, both approved by Dr. Spencer Smith as part of our problem statement (see ProblemStatement.tex, Section 4):

1. **Usability Testing.** We conducted a formal usability study with 5 participants (P1–P5) across 16 tasks covering account management, lobby interaction, core gameplay in both decimal and dozenal modes, scoring comprehension, and auxiliary features. We collected quantitative metrics (task completion rates, time on task, error counts, SUS scores) and qualitative data (think-aloud transcripts, debrief notes). The mean SUS score was 66.5, slightly below the industry average of 68, primarily due to participants with no prior non-decimal experience. Key findings included the need for visible illegal-move feedback (Q3 scored 2.20/5.00), dozenal onboarding tutorials, and increased rulebook discoverability. Seven prioritized recommendations were produced. Full details are in Usability Report.

2. **Design Thinking.** We documented a structured Design Thinking process that helped us define the core problem, understand our users, and choose prototype directions based on empathy and evidence rather than assumptions. This extra delivered value by shaping the final game's user-centered design: it reduced complexity, made our design problem and

stakeholder needs explicit, guided our interface design decisions with real user needs, and helped us prioritize features that supported learning without overwhelming gameplay. View our Design Thinking document here.

3 Design Iteration (LO11 (PrototypeIterate))

Our design evolved through four major iterations: Proof of Concept (POC), Revision 0 (Rev0), Revision 1 (Rev1), and the Final update. Each iteration represented a fundamental shift in scope and quality, driven by feedback from the instructor, our supervisor Prof. Rapoport, and our own usability testing.

3.1 POC: Command-Line Prototype

The POC was a purely text-based prototype with no graphical user interface. It ran entirely in command-line windows: two local terminal instances served as the two players. Players typed commands such as `play 7H` (7 of Hearts) to play a card, and the game state was rendered as text output in the console. The POC supported only the decimal numeral system and had no animations, no card graphics, and no networked multiplayer.

The purpose of the POC was to validate the core risk identified in our Development Plan: whether the sum-rule mechanic (two card ranks must sum to "10" in the active base) could be correctly implemented and whether the turn-based game loop was sound. The POC successfully demonstrated that the fundamental game logic worked, and the instructor confirmed that the risk was adequately addressed (POC scored 73.33%, Nov 28, 2025).

Prior to the POC, we held our first supervisor meeting with Prof. Rapoport on October 11, 2025 (Issue #43), where we discussed SRS and Hazard Analysis drafts, proposed counting rules (bonus points for correct arithmetic answers vs. penalties vs. game-stopping until correct), and explored what a dozenal playing deck would look like. Prof. Rapoport noted that virtual card assets were available, suggested we play the game in person to evaluate it, and confirmed that modifying the original Crazy Eights rules was acceptable since the primary goal was teaching dozenal arithmetic.

After the POC demo, we met Prof. Rapoport again on November 26, 2025 (Issue #133). He identified four features to prioritize: (1) arithmetic problems triggered by wildcards, (2) bonus points to encourage counting, (3) a timer on turns (potentially displayed in the active number base), and (4) multiplication operations. These priorities shaped the Rev0 and Rev1 feature roadmap.

3.2 Rev0: Ground-Up GUI Rewrite

Rev0 was a complete rewrite that shared no code with the POC. The decision to start from scratch was deliberate: the POC's command-line architecture had no path to a web-based GUI, and the technology stack (React, Node.js, Socket.io) required an entirely different project structure.

On January 22, 2026, we held an in-person supervisor meeting (Issue #182) where Ammar and Prof. Rapoport played 1–2 rounds of the game using a physical dozenal card deck. Prof. Rapoport reviewed the GUI design sketches and the Design Document Rev0. The meeting confirmed that the new gameplay features "work fine and make the game more interesting" and that a base conversion module should be planned, though only dozenal would be implemented initially.

Rev0 introduced a graphical card game interface, but it was still rough in many respects:

- The GUI was primitive with no transition animations and numerous visual bugs.
- The game only supported local play; there was no deployment to an external server.
- There was no real database, no user authentication or login system, and no persistent data.
- The lobby flow was unclear and confusing, making it difficult for players to start a match.
- Many core features were incomplete or missing entirely.

The instructor's Rev0 Demo feedback (Feb 13, 2026, scored 14/21) was critical. The feedback stated that "Rev0 is intended to be a complete implementation" and identified specific gaps: authentication not set up, Unicode not displaying correctly, development logs visible, actual card art missing, and no support for concurrent games. The difficulty score of 1.5/3 indicated the project was perceived as too easy at that stage. Team Communication scored only 1/3, highlighting coordination problems.

On February 5, Prof. Rapoport provided feedback via Discord: "The most important thing is to get the game to complete in dozenal, including the arithmetic questions. Leave the timer out for now." He also asked whether card sorting should be automatic or user-controlled, and requested weekly progress updates.

The pivotal meeting was on February 12, 2026, where all four team members demoed the Rev0 updates to Prof. Rapoport. After playing a full round of dozenal, the meeting produced a detailed list of 14 changes (documented by Ammar in Issue #263). Key items included: starting hand should be 7 cards not 5, the "10" card (not Dek) should be the wildcard, arithmetic operations should be triggered by face cards with suits determining the operation (in dozenal, face card J,Q,K trigger random arithmetic operation, C triggers the options to choose specific operation), answers should be typed not selected from multiple-choice, both players should compete on challenges simultaneously, the

goal should increase from 50 to 100, the pass button should be removed (automatic pass instead), and the score bug (displaying in decimal during dozenal games) must be fixed.

3.3 Rev1: Infrastructure and Feature Overhaul

Rev1 was the most substantial update, addressing nearly all of the Rev0 feedback and adding major new subsystems:

Infrastructure and deployment:

- Deployed the game on AWS EC2 instances, making it publicly accessible for the first time.
- Implemented a full PostgreSQL database system for persistent data storage.
- Built a complete login system with registration, authentication, and session management.
- Added a guest mode allowing users to play without creating an account.
- Implemented user profiles with match history, win rate tracking, recent 5 game histories, password changes, and nickname customization (separate from username).

Gameplay and logic improvements:

- Simplified and optimized lobby behavior: players can now view a list of all game rooms created by other players and join directly by clicking. Players can freely leave rooms before a match begins, and empty rooms are deleted automatically.
- Added a collapsible in-game chat system for player communication during matches.
- Added an admin mode accessible via admin credentials, allowing administrators to search for users and game rooms, change passwords, clear match histories, delete game rooms, and force logout.
- Implemented the arithmetic challenge system per Prof. Rapoport's feedback: suits determine operations (spades $\rightarrow$ addition, hearts $\rightarrow$ subtraction, diamonds $\rightarrow$ multiplication, clubs $\rightarrow$ division) with typed answers.

Bug fixes:

- Fixed a bug where the frontend username state reverted on page reload despite having a valid authentication token.

- Fixed server-side session clearing on manual logout.
- Fixed the arithmetic operation function, which was incorrectly calculating results in decimal even during dozenal games.
- Fixed incorrect bonus point awards from arithmetic challenges.
- Fixed the win condition goal being displayed as 100 in decimal rather than in the active base.

3.4 Final Update: Polish and User Experience

The final update was driven by three sources: usability testing findings, remaining instructor feedback from the Final Demo (Apr 1), and a detailed email from Prof. Rapoport on April 3 after he played the game independently. Prof. Rapoport noted that "the game has made many excellent advances, especially in the implementation of the arithmetic questions in dozenal" but identified several remaining issues: only 7 cards dealt instead of the required 8, auxiliary tables using X and E instead of the rotated 2 and 3 symbols, hand count displayed in decimal, missing round transition prompt, nearly invisible suit icons after wildcard plays, and the game not ending when a player's score exceeded 100 dozenal.

Core gameplay changes:

- Initial hand size increased from 7 to 8 cards. Hand count displayed in dozenal during dozenal games.
- Win condition changed from "empty your hand" to "reach 100 points first" (displayed in the active base).
- Implemented hand card sorting by suit and rank for better organization.
- Added a manual pass button (⇒ PASS), replacing the previous auto-pass-after-3-draws mechanic, giving players more control.
- Implemented hand card sorting function with smooth animation. (by suit/rank)
- User input for arithmetic operation challenge changed from typing to clicking numpad on screen. (Rotated 2 and 3 can not be typed directly, using X or E as alternatives may cause inconsistency)

Error feedback and UX improvements (driven by usability testing): Our usability study found that illegal-move feedback was effectively invisible (only shown in a collapsible activity log). The custom Likert question on error feedback (Q3) scored only 2.20/5.00, the lowest of all questions. In direct response:

- Implemented toast notifications that slide in from the top of the screen for all errors (login failures, illegal moves, connection issues, etc.).
- Added a hand-shake animation on invalid card plays, providing immediate visual feedback.
- Moved activity logs to a small button in the top-right corner, de-emphasizing them in favor of the new toast system.
- Improved the algorithm for generating arithmetic challegenge questions (previous multiplication/division such as 82x78 in decimal was too hard)

Arithmetic challenge overhaul:

- Added a 60-second countdown timer for arithmetic operations.
- Changed to a one-attempt lockout system with dual-player competition format.
- Added a challenge history panel for reviewing past results.

Visual and animation improvements:

- Added animated game background with 12 floating card suit symbols.
- Enhanced turn indicator: the active player's turn now displays a glowing green border with animated glow pulse and sweeping shimmer effect; the opponent's turn is muted and dimmed.
- Made the wildcard chosen-suit color more visible.
- Added a prominent mode selector for switching between Decimal, Dozenal, and Octal.
- Lobby expanded to fullscreen on desktop.
- Added show/hide toggle button for password fields.

Educational content improvements:

- Hint tables rewritten in HTML with proper dozenal turned-digit symbols instead of image-based fallback characters, improving accessibility and load times.
- Added experimental Octal mode (base-8), expanding the educational scope beyond decimal and dozenal, proved the potential of supporting different number bases.

Known remaining limitations:

- Sound effects are not implemented.
- Customizable card back themes were not implemented, only several built-in themes can be selected.
- Password encryption uses basic hashing rather than a production-grade algorithm.
- Network security hardening is incomplete.

4 Design Decisions (LO12)

Our project's architecture, user interface, and technical implementation were heavily influenced by specific limitations, assumptions, and constraints.

Limitations: Our primary limitation was the restricted development time and team size (four members without extensive prior real-time game development experience). This influenced our decision to adopt a full-stack JavaScript environment (React and Node.js) rather than introducing a steeper learning curve with tools like Unity or C++. By staying within the JavaScript ecosystem, we could reuse logic between the client and server. Additionally, deployment limitations (a single AWS EC2 instance budget) guided us to use Docker containers, ensuring our database, backend, and frontend could be reliably hosted and interconnected without complex cloud orchestration.

Assumptions: We assumed that our target users would have basic computer literacy but little to no prior background in dozenal (base-12) arithmetic. This assumption heavily shaped our user interface decisions: we prioritized making the educational aspects distinct and helpful. We designed accessible, HTML-based hint tables with proper turned-digit symbols, substituted manual typing with an on-screen numpad for answering arithmetic challenges to avoid confusion over non-standard characters, and provided clear visual feedback (like the screen shakes and toast notifications) to quickly correct user mistakes. Furthermore, we assumed an always-on, relatively stable internet connection, which justified using WebSockets (Socket.io) for real-time synchronization over a polling HTTP approach.

Constraints: Our most rigid constraint was pedagogical correctness. The final product had to serve as an educational tool for the Dozenal Society, meaning standard game mechanics had to be modified to enforce math skills. For example, traditional Crazy Eights allows arbitrary wildcard play; however, constrained by the goal of math education, we required a mathematical operation challenge (addition, subtraction, multiplication, division depending on suit) triggered by face cards. This requirement heavily constrained our game loop logic, forcing us to build a robust server-side state machine capable of freezing the

game, entering an arithmetic challenge phase, syncing dual-player inputs, applying countdown timers, and evaluating results correctly within the arbitrary numerical base.

5 Economic Considerations (LO23)

Market Potential and Target Audience: The immediate market for the Crazy 1-0s game includes educational institutions, mathematics hobbyists, and advocates of alternative numeral systems (such as the Dozenal Society of America and the Dozenal Society of Great Britain). While the broader educational games market is substantial, the niche for base-12 arithmetic is highly specialized. The potential user base currently comprises tens of thousands of math enthusiasts and educators seeking novel tools to teach base-operations.

Development Cost and Economics: To estimate the cost to produce a sellable version of this software, we consider the team's labor. Assuming 4 software engineers worked an average of 15 hours per week over a 28-week academic cycle, totaling 1,680 hours. At a standard intermediate developer rate of $40/hour, the labor cost equates to roughly $67,200. Ongoing infrastructure costs (AWS hosting, database instances, and domain registration) are extremely low at roughly $20-$30 per month.

Revenue Model and Marketing Strategy: Given the educational and niche nature of the project, a traditional retail model (e.g., selling the game for $15) would likely face significant friction, as schools and hobbyists typically prefer free or low-cost barrier entry tools. Instead, the game operates effectively as an Open Source or Free-to-Play application. To monetize without restricting the core educational value, a freemium model could be adopted: the base game remains free, but users could purchase cosmetic additions (e.g., customized card backs, animated backgrounds, unique avatars) or schools could pay a subscription for premium analytics (e.g., classroom dashboards tracking student progress in arithmetic challenges).

Attracting Users: As an educational open-source project, our marketing strategy relies heavily on community engagement rather than paid advertising. We would attract users by:

- Partnering with math influencers on platforms like YouTube and forums (e.g., Reddit's r/math).
- Presenting the application at educational technology conferences and Dozenal Society annual meetings.
- Utilizing Search Engine Optimization (SEO) for terms related to "dozenal arithmetic", "base-12 games", and "alternative math education".
- Relying on word-of-mouth through classroom adoption.

6 Reflection on Project Management (LO24)

6.1 How Does Your Project Management Compare to Your Development Plan

We largely followed the Development Plan, with some deviations:

- **Team Meeting Plan:** The Development Plan had "as required/appropriate" with one or two meetings before major deliverables. In practice, we held very few formal team meetings (3 before POC, 2 between POC and Rev0, 2 between Rev0 and Final). Most coordination happened asynchronously through Discord, which was far more frequent than planned MS Teams sessions. However, the Development Plan did anticipate that "most work will be done and delegated through GitHub project management and via Discord server," so our actual practice was broadly consistent with the plan.

- **Team Communication Plan:** We used all five planned channels (GitHub, Discord, Email, MS Teams, In-Person), but the balance shifted heavily toward Discord and GitHub Issues. MS Teams was used sparingly, and most design discussions occurred in Discord voice channels rather than scheduled MS Teams calls. Email was used only for supervisor correspondence. Overall, the asynchronous-first communication style worked well for our team's schedules.

- **Workflow Plan:** We followed the planned Git workflow (feature branches, pull requests, at least one reviewer approval before merging) for the majority of the project. GitHub Issues and the Kanban board were used consistently throughout. However, the branch discipline was not perfect: there were occasional direct pushes to shared branches, particularly close to deadlines.

- **Technology:** We used nearly all the planned technologies: React (frontend), Node.js (backend), PostgreSQL (database), Socket.io (real-time networking), GitHub Actions (CI/CD), and LaTeX (documentation). The main deviation was switching from Jest to Vitest as our testing framework, as Vitest integrates more naturally with the Vite build tool we adopted. We also used Docker for deployment, which was not originally listed but became essential for hosting on AWS EC2.

6.2 What Went Well?

[What went well for your project management in terms of processes and technology? —TPLT]

- **GitHub Issues and Kanban Board:** Our use of GitHub Issues for tracking every task from lectures and meetings to feature implementation and bug fixes provided excellent traceability. By the end of the project,

the team had authored and closed hundreds of issues across all phases. The Kanban board (Backlog / In Progress / Review / Done) gave the team a clear visual of project status at any time.

- **Supervisor Engagement:** Having Ammar as a dedicated Team Liaison ensured regular, productive meetings with Prof. Rapoport. We held 8 supervisor meetings before the POC, 5 between POC and Rev0, and 3 between Rev0 and the Final Demo. Each meeting produced concrete, actionable feedback (documented in GitHub Issues), and the team implemented the vast majority of suggestions. Prof. Rapoport's final email noted that "the game has made many excellent advances, especially recently," validating this feedback loop.

- **CI/CD Pipeline:** The GitHub Actions pipeline (automated builds, Vitest test execution, and LaTeX compilation on every PR) caught integration issues early and enforced a baseline of code quality. This was a direct response to instructor feedback at the POC that CI/CD should be implemented, and it served the team well from Rev0 onward.

- **Discord for Asynchronous Coordination:** While informal, Discord proved to be our most effective communication tool. Quick questions, design debates, and status updates happened in near real-time without the overhead of scheduling formal meetings. This flexibility was critical given that three of the four team members had packed course schedules.

- **Iterative Development:** The project naturally followed an iterative cycle (POC $\rightarrow$ Rev0 $\rightarrow$ Rev1 $\rightarrow$ Final), with each iteration incorporating feedback from the previous demo. This structure kept the scope manageable and ensured continuous improvement.

6.3 What Went Wrong?

- **Uneven Workload Distribution:** Commit and issue data across all three team contribution reports reveal a persistent imbalance. For example, between POC and Rev0, Ruida had 25 commits while Jiaming had 2. Between Rev0 and the Final, Jiaming had 13 commits while Ammar had 1. While commit counts do not perfectly reflect effort (different members used different commit styles, and some work like game design or stakeholder meetings does not generate commits), the imbalance was real and occasionally led to bottlenecks when key contributors were unavailable.

- **Insufficient Formal Team Meetings:** We held very few structured team meetings throughout the project. While Discord filled the gap for day-to-day coordination, the lack of regular synchronous check-ins contributed to the "Team Communication" score of only 1/3 at the Rev0 Demo. Important design decisions were sometimes made in fragmented Discord conversations rather than deliberate, documented discussions.

- **Late Integration and Incomplete Rev0:** The Rev0 demo was intended to be a "complete implementation" per the instructor's expectations, but we delivered a product with no authentication, no deployment, missing card art, and visible development logs. This gap between expectations and delivery reflected insufficient planning for the Rev0 milestone and underestimation of the integration effort required to combine separately developed frontend and backend components.

- **Documentation Feedback Not Addressed Promptly:** Some feedback from TA document discussions (SRS peer review comments, Design Document Rev−1 feedback) was not incorporated until much later, sometimes only when the next revision's deadline forced the changes. A more disciplined approach to addressing feedback within a fixed window would have improved document quality earlier.

- **Branch Discipline Lapses:** Despite the workflow plan requiring all changes to go through pull requests with at least one approval, there were instances of direct pushes to shared branches near deadlines. These bypassed the review process and occasionally introduced bugs that had to be fixed later.

6.4 What Would you Do Differently Next Time?

- **Establish a Fixed Weekly Meeting:** Rather than meeting "as required," we would schedule a mandatory 30-minute weekly stand-up (even if virtual). This would provide a regular synchronization point, catch miscommunications early, and demonstrate consistent team communication to evaluators.

- **Define Explicit Per-Sprint Ownership:** Instead of allowing specialization to emerge organically, we would assign balanced workloads at the start of each sprint (two-week iterations), including cross-functional tasks (a backend-focused developer takes a frontend task). This would reduce bus-factor risk and ensure all members gain experience across the full stack.

- **Set Internal Deadlines Before External Ones:** We would establish internal "code freeze" and "feature complete" deadlines at least one week before the actual demo or submission date. This buffer would allow time for integration testing, documentation updates, and rehearsal — all of which were rushed in this project.

- **Create a Feedback Backlog with SLAs:** Each item of feedback (from TAs, instructor, supervisor, or peers) would be logged as a GitHub Issue immediately, assigned an owner, and given a target resolution date (within two weeks of receipt). This would prevent feedback from being lost or deferred indefinitely.

- **Invest More in Automated Testing Early:** Our test suite grew mostly after Rev0. Writing tests alongside feature development from the POC stage onward would have caught bugs earlier (the decimal-vs-dozenal arithmetic bug) and built confidence in refactoring.
- **Rehearse Demos:** Both the Rev0 and Final Demo received presentation-related feedback (low energy, too much time on roadmap, insufficient rehearsal). We would schedule at least one full mock demo with timing and feedback before each evaluation.

7 Ethics and Equity (LO18)

Our team applied equity and universal design by making the game interface accessible and fair for all stakeholders rather than optimizing only for a narrow expert audience. We designed the game so that:

- everyone sees the same clear game state,
- new learners are not penalized by hidden rules or confusing notation,
- mathematical concepts are supported with visible scores and help,
- the UI supports both quick gameplay and instructional use.

That means the game was built to support diverse users fairly, with a consistent, low-barrier experience for all.

Equity to different stakeholder groups

Client: Professor Rapoport gets a teachable, mathematically correct game with visible dozenal notation and clear educational support.
Client: Dr. Smith gets a stable, predictable system whose behavior is easy to observe and evaluate.
Customer: Mathematicians get explicit dozenal scoring and dual-notation support that preserves accuracy while making the math understandable.
Customer: Educators get a game that can be used in class because the interface is clear, the rules are visible, and learning support is built in.
Customer: Students get a low-barrier experience with clear actions, visible feedback, and helpful hints, so the game is fair even for learners new to alternative number systems.

8 Reflection on Capstone

8.1 Which Courses Were Relevant

- **SFWRENG 2AA4 — Software Design I:** Git workflow, Kanban board management, object-oriented design patterns. These were foundational for our project's version control strategy and modular architecture.
- **SFWRENG 3A04 — Software Design III (Large System Design):** UML diagrams, software architecture documentation, module decomposition. Directly applicable to our MG and MIS documents, and the 20-module architecture.
- **SFWRENG 3RA3 — Software Requirements:** Requirements elicitation, SRS writing, GitHub Issues for tracking. Essential for our SRS document and the systematic approach to requirements traceability.
- **SFWRENG 3S03 — Software Testing:** Test case design, automated testing frameworks, coverage analysis. Applied directly in our Vitest unit test suite and V&V process.
- **SFWRENG 4HC3 — Human Computer Interfaces:** User-centered design, usability testing methodology, SUS questionnaire design. Critical for our two extras (Usability Testing and Design Thinking) and for designing an interface that balances education with gameplay.
- **SFWRENG 4C03 — Computer Networks and Security:** Network protocols, WebSocket fundamentals, security best practices. Applied in our real-time multiplayer architecture (Socket.io), TLS configuration, and authentication system.

8.2 Knowledge/Skills Outside of Courses

- Deepened understanding of React component architecture, state management, and CSS animation design for card interactions. Learned through official React documentation and iterative prototyping.
- Gained hands-on experience with Node.js, Express, and PostgreSQL database management. Built the API layer and repository module from scratch, learning through online tutorials and course project repositories.
- Learned game state machine patterns, turn-based engine architecture, and real-time synchronization techniques. These were not covered in coursework and required independent research into game development best practices.

- Acquired domain knowledge in dozenal arithmetic, its history, and pedagogical approaches to teaching alternative bases. Also developed skills in stakeholder communication through regular meetings with Prof. Rapoport and in preparing demonstration presentations.

- While some members had prior exposure, producing the volume of documentation required for this capstone (SRS, MG, MIS, VnV Plan/Report, Hazard Analysis, Usability Report, Design Thinking Report, and this Reflection) required significant LaTeX proficiency that went beyond course expectations. Collaborative LaTeX editing and resolving merge conflicts in .tex files were learned on the job.

- Learned web deployment and devops, docker container techniques for web hosting.

- Building a real-time multiplayer game with Socket.io required understanding WebSocket lifecycle management, room-based event routing, server-authoritative state synchronization, and reconnection handling. None of our courses covered real-time web application development in depth.